

使用 BigQuery 和 ADK 建構事件導向的資料代理

來源：<https://codelabs.developers.google.com/bigquery-adk-event-driven-agents?hl=zh-tw>

步驟數：12

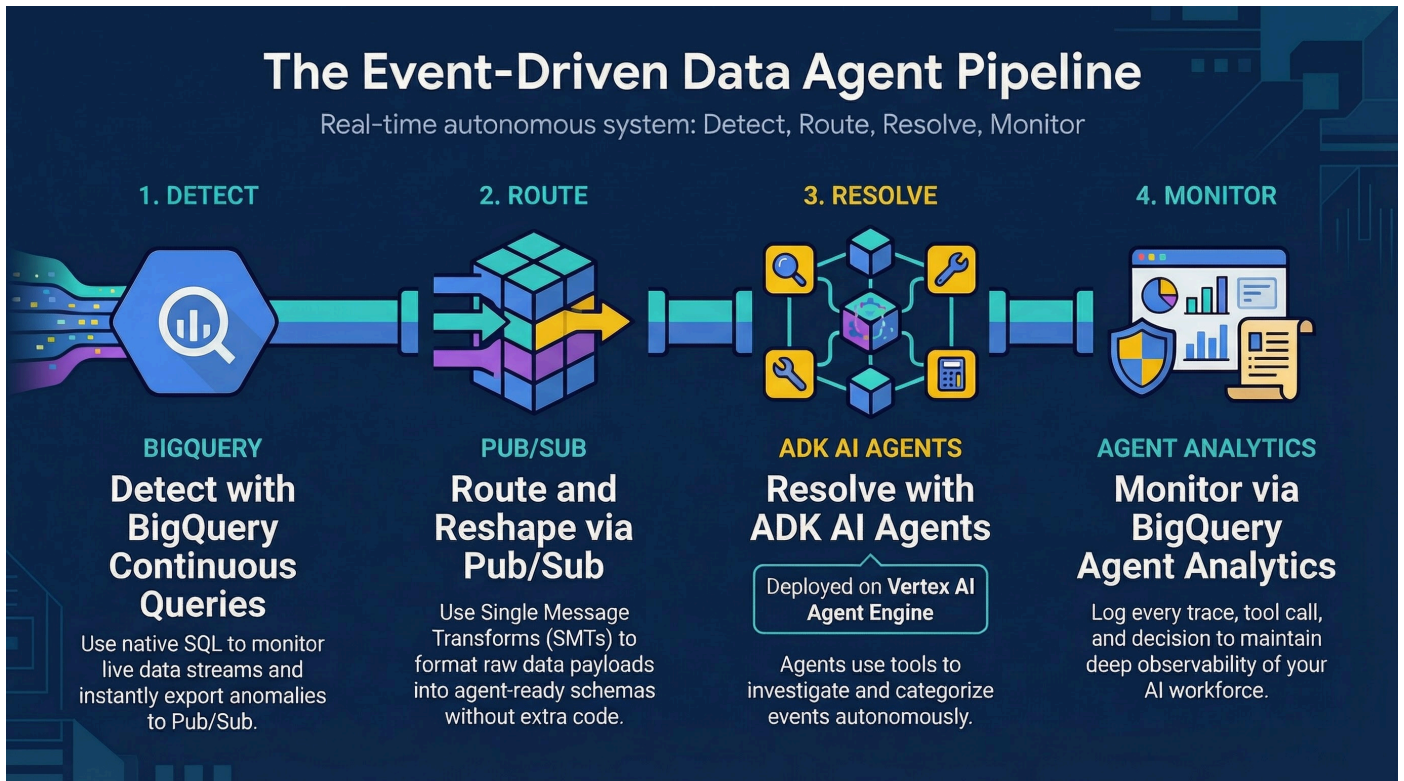
使用 BigQuery 持續查詢、Pub/Sub 和 Agent Development Kit (ADK)，建構以事件為核心的資料代理。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 準備環境](#)
4. [4. 檢查 ADK 代理](#)
5. [5. 部署 ADK 代理](#)
6. [6. 測試 ADK 代理](#)
7. [7. 設定 BigQuery 持續查詢，將案件升級資訊串流至 Pub/Sub](#)
8. [8. 建立推送訂閱項目](#)
9. [9. 產生事件](#)
10. [10. 在 BigQuery 中分析代理程式效能](#)
11. [11. 清除所用資源](#)
12. [12. 恭喜](#)

1. 簡介

在本程式碼實驗室中，您將建構事件導向架構，結合 BigQuery 持續查詢、Pub/Sub 和使用 Agent Development Kit (ADK) 建構的詐欺調查員代理，並將其代管於 Vertex AI Agent Engine。



您將設定管道，持續查詢即時零售交易中的異常狀況 (例如「不可能的旅行」)，並將這些可疑事件匯出至 Pub/Sub 主題，然後觸發 ADK 代理程式，個別評估及回應每項異常狀況。

學習內容

- 準備含有交易範例資料的 BigQuery 環境
- 建立 BigQuery 持續查詢，偵測即時異常狀況
- 設定含有單一訊息轉換 (SMT) 的 Pub/Sub 主題和訂閱項目
- 將 ADK 代理提取、設定及部署至 Vertex AI Agent Engine
- 串流交易資料，驗證代理程式是否收到並處理升級要求

軟硬體需求

- 網路瀏覽器，例如 [Chrome](#)
- 已啟用計費功能的 Google Cloud 雲端專案
- 存取 Google Cloud Shell

本程式碼研究室適合熟悉 BigQuery 和基本 Python 的中階開發人員。

本程式碼研究室建立的資源費用應低於 \$2 美元。

預計時間：完成本程式碼研究室約需 60 分鐘。

注意：本程式碼實驗室會引導您建構事件導向管道，但這是簡化版的 Cymbal Bank：在 2026 年 Google I/O 大會上展示的代理式資料雲端。

如要全面瞭解 Cymbal Bank「行動系統」架構，請參閱 [Cymbal Bank 示範總覽](#)。本文詳細說明完整的詐欺防範工作流程，從 BigQuery 中的高速篩選到代理功能動作，都能大規模執行。

步驟 2 · 約 5 分鐘

2. 事前準備

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 按一下 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 連至 Cloud Shell 後，請驗證您的驗證：

```
gcloud auth list
```

3. 確認專案已設定完成：

```
gcloud config get project
```

4. 如果專案未如預期設定，請設定專案：

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

設定專案 ID

執行下列指令，擷取有效的 Google Cloud 專案 ID，並將其儲存為環境變數，以便在本程式碼研究室中使用：

```
export PROJECT_ID=$(gcloud config get-value project)
```

取得程式碼

執行這項指令來複製存放區，並只下載包含 ADK 代理程式和設定指令碼的目標 `event_driven_agents_demo` 資料夾：

```
git clone --depth 1 --filter=blob:none --sparse  
https://github.com/GoogleCloudPlatform/devrel-demos.git temp-repo && cd temp-repo && git  
sparse-checkout set data-analytics/event_driven_agents_demo && cd .. && mv temp-repo/data-  
analytics/event_driven_agents_demo . && rm -rf temp-repo
```

請前往 `event_driven_agents_demo` 目錄：

```
cd event_driven_agents_demo
```

開啟 Cloud Shell 編輯器後，您應該會看到複製的存放區結構：



CLOUD SHELL

Editor

File

Edit

Selection

View

Go

...



EXPLORER

...



✓ **CATELYNTARLY_548490**

✓ event_driven_agents_demo

✓ agent

> adk_agent_app

 agent_runner.py

 deploy_agent_script.py

≡ requirements.txt

✓ setup

\$ cleanup_env.sh

 continuous_query.sql

 customer_profiles.csv

 load_historical_data.py

JS pubsub_to_adk_transform.js

\$ setup_env.sh

! transform.yaml

✓ simulator

{ } bulk_events.json



 generate_events.py

 test_event.json

 README-cloudshell.txt

步驟 3 · 約 5 分鐘

3. 準備環境

您將使用存放區中提供的設定指令碼，準備 Google Cloud 環境。這個指令碼：

- 佈建 *Google Cloud Storage* 值區，用於暫存 Agent Developer Kit (ADK)
- 建立 *CONTINUOUS Enterprise BigQuery* 預留項目，用於處理查詢
- 設定 *BigQuery* 資料集，並載入初始 *customer_profiles* 資料
- 設定 *IAM* 權限，並將必要角色授予 ADK 代理程式服務帳戶

從 Cloud Shell 執行指令碼：

```
chmod +x setup/setup_env.sh  
./setup/setup_env.sh
```

步驟 4 · 約 5 分鐘

4. 檢查 ADK 代理

現在要將 ADK 代理程式碼部署至 Vertex AI Agent Engine。請先執行這項操作，確保代理程式已部署完畢，並準備好處理升級要求，再開始串流資料。

```
cd agent
```

瞭解 ADK (Agent Development Kit) 代理程式碼

核心代理邏輯是在 `adk_agent_app/agent.py` 中定義。

我們建構的代理會使用 Gemini 2.5 Flash，自主調查異常警報。代理程式會分析快訊酬載、從 BigQuery 擷取顧客記錄，並透過網路搜尋驗證商家詳細資料，然後將交易分類為 `FALSE_POSITIVE` (合法交易) 或 `ESCALATION_NEEDED`。

```
# Excerpt from agent/adk_agent_app/agent.py
investigation_agent = Agent(
    model="gemini-2.5-flash",
    name="Fraud_Investigation_Agent",
    description="Expert fraud analyst agent that autonomously investigates alerts...",
    instruction=(
        "You are an expert fraud investigator for Cymbal Bank. "
        "Your goal is to investigate financial transaction alerts, "
        "determine if they are fraudulent, and take appropriate action. "
        "Calculate risk, assess the logic_signals provided in the input, "
        "query the database for past transactions, and search the merchant..."
    ),
    tools=[
        bigquery_toolset,
        google_search,
    ],
)
```

這個代理程式配備兩種不同的工具：

1. **BigQueryToolset**：允許代理程式自主查詢 `cymbal_bank` 資料集，以尋找其他交易記錄。
 2. **google_search**：允許服務專員在網路上搜尋，調查商家的聲譽並驗證其合法性。
-

步驟 5 · 約 10 分鐘

5. 部署 ADK 代理

執行下列指令，安裝部署代理程式所需的 Python 套件 (`google-cloud-aiplatform` 、 `google-adk` 等)：

```
pip install -r requirements.txt
```

執行下列指令，動態產生含有特定專案 ID 的 `.env` 檔案，部署代理程式時會用到這個檔案：

```
cat <<EOF > .env
PROJECT_ID=$PROJECT_ID
LOCATION=us-central1
STAGING_BUCKET=gs://$PROJECT_ID-adk-staging
SERVICE_ACCOUNT=adk-agent-sa@$PROJECT_ID.iam.gserviceaccount.com
BIGQUERY_DATASET=cymbal_bank
GOOGLE_GENAI_USE_VERTEXAI=1
EOF
```

注意：系統預設會隱藏 `.env` 檔案。如要在 Cloud Shell 編輯器中查看，請在檔案總管檢視畫面中啟用「顯示隱藏檔案」。

現在執行下列指令，將代理部署至 Vertex AI Agent Engine：

```
python deploy_agent_script.py
```

注意：`deploy_agent_script.py` 會初始化 `BigQueryAgentAnalyticsPlugin`，自動將追蹤資料和代理程式工具用量記錄到 BigQuery 的 `agent_events` 資料表。

這項作業要幾分鐘才能完成。畫面會顯示類似以下的輸出：

```
Deploying Agent...
Deployed Resource Name:
projects/<YOUR_PROJECT_ID>/locations/<REGION>/reasoningEngines/...

=====
Pub/Sub Push Endpoint URL:
https://<REGION>-
aiplatform.googleapis.com/v1/projects/<YOUR_PROJECT_ID>/locations/<REGION>/reasoningE
ngines/...:streamQuery
=====
```

執行下列指令，將部署的代理程式端點網址儲存至名為 `agent_endpoint.txt` 的本機檔案：

```
export AGENT_ENDPOINT=$(cat agent_endpoint.txt)
```

稍後建立 Pub/Sub 推送訂閱項目時，我們會使用這個網址。

6. 測試 ADK 代理

產生直播活動前，請先測試 Agent Engine 中的 ADK 服務專員是否能正確處理手動升級。

1. 前往 Google Cloud 控制台的「Vertex AI Agent Engine」頁面。
2. 按一下已部署的代理程式名稱 (`Cymbal Bank Fraud Assitant`)。
3. 前往「Playground」分頁，直接與代理互動。
4. 在即時通訊介面中，貼上以下模擬的 JSON 事件有效負載，模仿代理程式從 Pub/Sub 接收的內容，然後按 Enter 鍵：

```
{
  "window_end": "2026-03-15T10:00:00Z",
  "user_id": "user_39175",
  "customer_name": "Jonathan Mckinney",
  "tx_count": 1,
  "total_window_spend": 15.0,
  "highest_value_merchant": "Google One Subscription",
  "highest_value_mcc": "5732",
  "contains_international_tx": false,
  "contains_untrusted_device_tx": false,
  "final_risk_score": 2,
  "logic_signals": {
    "is_impossible_travel": false,
    "has_security_mismatch": false,
    "is_high_velocity": false
  }
}
```

確認代理程式評估交易，並在 Playground 視窗中回應 `FALSE POSITIVE` 評估結果：

#9 

✓ Search_Agent

#10 

⚡ log_final_decision

#11 

✓ log_final_decision

#12 

The alert for user user_39175 with merchant "YouTube" is a **FALSE_POSITIVE**. There are no historical transactions for this user in the past 7 days, and YouTube is a legitimate merchant. The transaction is not considered fraudulent.

Type a Message...

⋮

步驟 7 · 約 10 分鐘

7. 設定 BigQuery 持續查詢，將案件升級資訊串流至 Pub/Sub

ADK 代理程式已部署完成，可以接收事件，現在請返回根目錄，建構其餘的管道：

```
cd ../../event_driven_agents_demo
```

1. 建立 Pub/Sub 主題

執行這項指令，建立 Pub/Sub 主題。這個主題會接收從 BigQuery 持續查詢匯出的異常狀況：

```
gcloud pubsub topics create cymbal-bank-escalations-topic
```

我們會在下一個步驟中建立這個主題的訂閱項目。

2. 執行 BigQuery 持續查詢

部署代理程式並準備好 Pub/Sub 主題後，即可啟動持續查詢，即時監控 `retail_transactions` 串流。這項查詢會偵測「不可能的旅行」異常狀況，並將快訊匯出至 Pub/Sub。

執行下列指令來啟動查詢：

```
sed -i "s/YOUR_PROJECT_ID/$PROJECT_ID/g" setup/continuous_query.sql

bq query \
  --use_legacy_sql=false \
  --continuous=true \
  --sync=false \
  --connection_property=service_account=adk-agent-sa@$PROJECT_ID.iam.gserviceaccount.com \
  "$(cat setup/continuous_query.sql)"
```

BigQuery 連續查詢會持續執行，並在來源資料表有新資料時發出結果。`EXPORT DATA` 陳述式會將相符的資料列直接傳送至 Pub/Sub 主題。持續查詢可能需要一兩分鐘才能完全啟動並開始處理資料。

終端機中應該會顯示輸出內容，指出持續查詢已順利啟動：

```
Successfully started query your-project-id:bqjob_r66189572226875ed_0000019d000000_1
```

8. 建立推送訂閱項目

現在代理程式已部署完成，且持續查詢正在執行中，您將建立「推送」訂閱項目，主動將主題中的任何新異常狀況訊息直接轉送至代理程式的 Webhook 網址。

為確保代理程式收到正確格式的資料，我們會使用**單一訊息轉換 (SMT)**。您可以透過 SMT 功能，直接在 Pub/Sub 中簡單修改訊息資料與屬性，然後再傳送給訂閱者。

以下是管道中的轉換運作方式：

- **UDF**：setup 目錄中的 transform.yaml 檔案包含 JavaScript 使用者定義函式 (UDF)，可處理訊息。
- **解開 BigQuery 資料包裝**：當 BigQuery 透過持續查詢將資料匯出至 Pub/Sub 時，會將 JSON 酬載包裝在外部物件中。
- **ADK 的格式**：UDF 會解開雙重編碼，並將酬載重新封裝為 Agent Engine streamQuery API 預期的嚴格格式。

執行下列指令，建立已套用 UDF 轉換的訂閱項目：

```
gcloud pubsub subscriptions create cymbal-bank-escalations-sub \
  --topic=projects/$PROJECT_ID/topics/cymbal-bank-escalations-topic \
  --message-transforms-file=setup/transform.yaml \
  --push-endpoint="$AGENT_ENDPOINT" \
  --push-no-wrapper \
  --push-auth-service-account="adk-agent-sa@$PROJECT_ID.iam.gserviceaccount.com" \
  --ack-deadline=600
```

您應該會看到確認訂閱項目已建立的輸出內容：

```
Created subscription [projects/your-project-id/subscriptions/cymbal-bank-escalations-sub].
```


9. 產生事件

最後，請執行 `generate_events.py`，將合成的「Impossible Travel」交易串流至 `cymbal_bank.retail_transactions` 資料表，測試端對端流程：

```
python simulator/generate_events.py
```

這項功能會使用我們稍早載入的顧客設定檔資料 (Karen Burton，居住國家/地區為美國)，模擬在澳洲 (AUS) 發生的一筆 \$2,500 美元的電子產品交易。

確認事件是否送達：等待約兩分鐘，讓持續查詢視窗化和 ADK 處理程序完成作業，然後檢查已部署的代理程式記錄，確認是否已處理觸發的 Pub/Sub 訊息。

[←](#) Cymbal Bank Fraud Assitant

[Service Configuration](#)

[Copy query URL](#)

[Copy identity](#)

Dashboard

Runtime Metrics

Traces

Sessions

Playground

Memories

Evaluation

<

🕒

Last 30 minutes

🔍

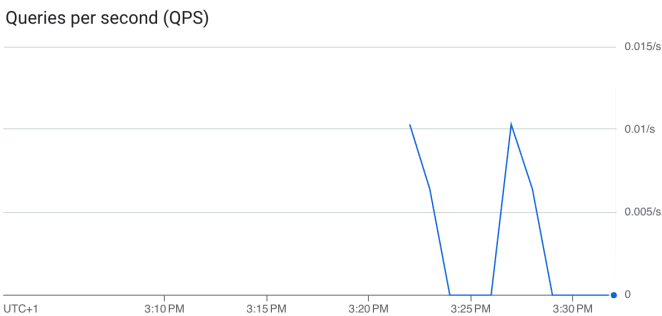
>

🗑️

Agent Engine Queries 🔗

This section describes the overall query performance. Use these metrics to identify performance anomalies or diagnose reliability issues.

Queries per second (QPS)



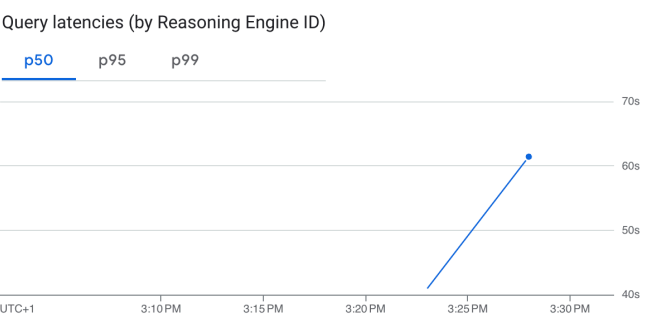
● request_count 4451536102527860736

Query latencies (by Reasoning Engine ID)

p50

p95

p99



● request_latencies 4451536102527860736

10. 在 BigQuery 中分析代理程式效能

前往 [BigQuery 控制台](#)，然後選取 `cymbal_bank` 資料集。選取 `agent_events` 資料表，然後按一下「預覽」：

☆ agent_events 🔍 Query 🗨️ Create conversation Open in ▾ 👤 Share ▾ 📄 Copy 🖼️ Snapshot 🗑️ Delete ↗️ Export ↻

Schema Details Preview Table Explorer Preview Insights Lineage Data Profile Data Quality

❗ Preview may not show all data being streamed to BigQuery. If you want to check for such data, run a SELECT statement over the table.

Row	timestamp	event_type	agent	session_id	invocation_id	user_id
1	2026-04-08 14:16:29.309937 UTC	USER_MESSAGE_RECEIVED	Fraud_Investigation_Agent	4629643654649085952	e-4687dcf3-ac3d-49f6-b748-bc...	vais-query-reasoning
2	2026-04-08 14:16:29.489503 UTC	INVOCATION_STARTING	Fraud_Investigation_Agent	4629643654649085952	e-4687dcf3-ac3d-49f6-b748-bc...	vais-query-reasoning
3	2026-04-08 14:16:29.489736 UTC	AGENT_STARTING	Fraud_Investigation_Agent	4629643654649085952	e-4687dcf3-ac3d-49f6-b748-bc...	vais-query-reasoning
4	2026-04-08 14:16:29.512135 UTC	LLM_REQUEST	Fraud_Investigation_Agent	4629643654649085952	e-4687dcf3-ac3d-49f6-b748-bc...	vais-query-reasoning
5	2026-04-08 14:16:32.719762 UTC	LLM_RESPONSE	Fraud_Investigation_Agent	4629643654649085952	e-4687dcf3-ac3d-49f6-b748-bc...	vais-query-reasoning
6	2026-04-08 14:16:32.907245 UTC	TOOL_STARTING	Fraud_Investigation_Agent	4629643654649085952	e-4687dcf3-ac3d-49f6-b748-bc...	vais-query-reasoning

Results per page: 50 ▾ 1 – 36 of 36 |< < > >|

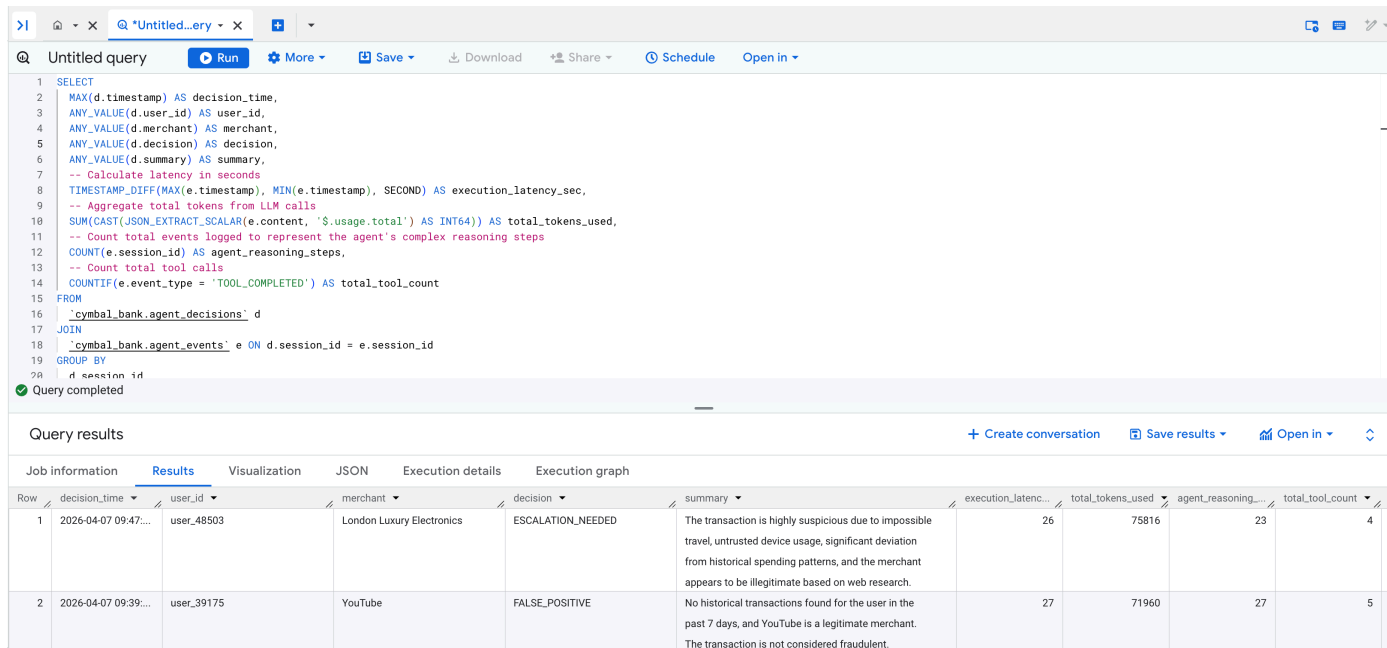
輸出內容會確認代理程式已成功分析「Impossible Travel」案件。

由於自主代理會在背景持續執行，因此可觀測性至關重要。代理程式會透過 ADK 外掛程式自動記錄執行追蹤記錄，並透過自訂工具記錄決策。

執行下列查詢，將代理的決策與 `agent_events` 資料表中擷取的延遲時間和權杖用量指標合併：

```
-- Create session-level metrics from detailed agent events
SELECT
  MAX(d.timestamp) AS decision_time,
  ANY_VALUE(d.user_id) AS user_id,
  ANY_VALUE(d.merchant) AS merchant,
  ANY_VALUE(d.decision) AS decision,
  ANY_VALUE(d.summary) AS summary,
  -- Calculate latency in seconds
  TIMESTAMP_DIFF(MAX(e.timestamp), MIN(e.timestamp), SECOND) AS execution_latency_sec,
  -- Aggregate total tokens from LLM calls
  SUM(CAST(JSON_EXTRACT_SCALAR(e.content, '$.usage.total') AS INT64)) AS total_tokens_used,
  -- Count total events logged to represent the agent's complex reasoning steps
  COUNT(e.session_id) AS agent_reasoning_steps,
  -- Count total tool calls
  COUNTIF(e.event_type = 'TOOL_COMPLETED') AS total_tool_count
FROM
  `cymbal_bank.agent_decisions` d
JOIN
  `cymbal_bank.agent_events` e ON d.session_id = e.session_id
GROUP BY
  d.session_id
ORDER BY
  decision_time DESC
```

您應該會看到類似下方的結果資料表：



Row	decision_time	user_id	merchant	decision	summary	execution_latency_sec	total_tokens_used	agent_reasoning_steps	total_tool_count
1	2026-04-07 09:47:...	user_48503	London Luxury Electronics	ESCALATION_NEEDED	The transaction is highly suspicious due to impossible travel, untrusted device usage, significant deviation from historical spending patterns, and the merchant appears to be illegitimate based on web research.	26	75816	23	4
2	2026-04-07 09:39:...	user_39175	YouTube	FALSE_POSITIVE	No historical transactions found for the user in the past 7 days, and YouTube is a legitimate merchant. The transaction is not considered fraudulent.	27	71960	27	5

無限可能：本 CodeLab 最後會將代理程式的決策記錄到 BigQuery，以進行視覺化。此外，事件產生器指令碼相對簡單，只會插入單一使用者的詐欺行為。請記住，代理程式工具只是 Python 函式。也就是說，隨著試用版擴展到更多用途或情境，代理程式可以與任何事物互動。

在正式環境中，您可以輕鬆擴充這個架構。除了記錄資料，您的代理程式還可以觸發 Webhook 來發出 Slack 或 Teams 管道的快訊、觸發 PagerDuty 事件、將最終判決寫入 Cloud Spanner 等低延遲資料庫，或是將新的 Pub/Sub 訊息發布至下游微服務，自動凍結遭盜用的信用卡！

11. 清除所用資源

如要避免系統持續向您的 Google Cloud 帳戶收取費用，請刪除在本程式碼研究室中建立的資源。

程式碼實驗室存放區包含清理指令碼，可自動刪除 Pub/Sub 部署作業、BigQuery 資料集、BigQuery 預留時段、Vertex Agent Engine 設定、Cloud Storage 暫存值區和 IAM 服務帳戶。

如果 BigQuery 持續查詢仍在執行，請透過 Google Cloud 控制台的 BigQuery UI 停止查詢。接著執行清除指令碼：

```
chmod +x setup/cleanup_env.sh
./setup/cleanup_env.sh
```

或者，如果您專為本程式碼研究室建立了專案，可以選擇刪除整個專案。

12. 恭喜

恭喜！您已使用 BigQuery、Pub/Sub 和 ADK 建構事件導向的資料代理管道。

目前所學內容

- 如何將 BigQuery 持續查詢中的異常狀況匯出至 Pub/Sub
- 如何將轉換後的 Pub/Sub 訊息傳送至 ADK 代理程式
- 如何在 Vertex AI Agent Engine 部署代理並與之互動

參考文件

- [BigQuery 持續查詢](#)
 - [Agent Development Kit \(ADK\) 說明文件](#)
 - [Pub/Sub 說明文件](#)
-